指标篇

《100天风控专家》

KS风险指标

出品：东哥起飞

解锁风控课程

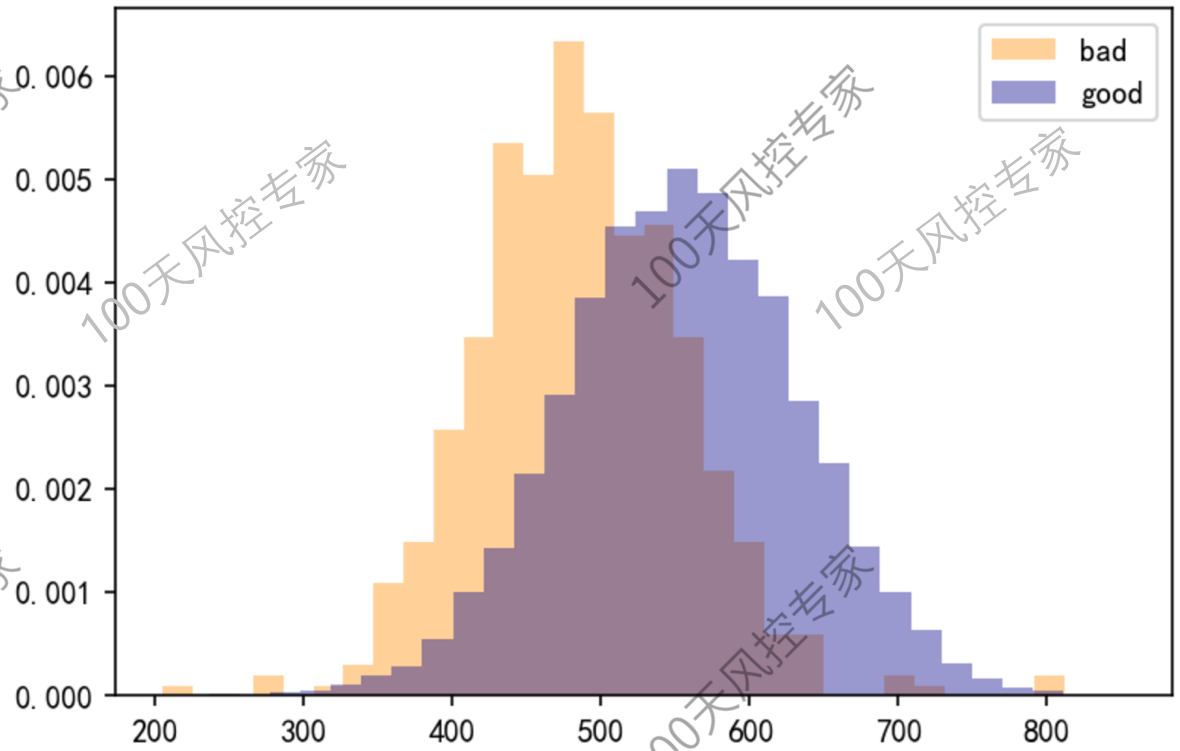
关注我的公众号

1. 什么是KS?

1）定义

KS（Kolmogorov-Smirnov）统计量由两位苏联数学家A.N. Kolmogorov和N.V. Smirnov提出。

在风控中，KS常用于评估模型或者变量的区分度，业内通用的定义为：**好坏客户累积分布差异的最大值。**



2. KS计算步骤

2）计算步骤

① 变量分箱，可以选择等频、等距、决策树或者自定义均可

② 计算每个分箱区间的好客户数（good_count）和坏客户数（bad_count）

③ 计算每个分箱区间的好/坏客户占比（goodpct/badpct）、累计好/坏客户数占比（cum_goodpct/cum_badpct）

④ 计算每个分箱区间累计好/坏客户数占比差的绝对值，得到KS

⑤ 在所有分箱的KS中取最大值，得到该变量最终的KS值

$$ks = \max\{|cum(bad\_rate) - cum(good\_rate)|\}$$

xgb_score	sum	bad_count	good_count	badpct	goodpct	cum_badpct	cum_goodpct	ks
(205.388, 448.898]	1000	150	850	30.00%	8.95%	30.00%	8.95%	21.05%
(448.898, 483.828]	1000	100	900	20.00%	9.47%	50.00%	18.42%	31.58%
(483.828, 508.842]	1000	70	930	14.00%	9.79%	64.00%	28.21%	35.79%
(508.842, 530.35]	1001	50	951	10.00%	10.01%	74.00%	38.22%	35.78%
(530.35, 550.93]	1001	45	956	9.00%	10.06%	83.00%	48.28%	34.72%
(550.93, 570.996]	998	35	963	7.00%	10.14%	90.00%	58.42%	31.58%
(570.996, 592.425]	1000	20	980	4.00%	10.32%	94.00%	68.74%	25.26%
(592.425, 618.716]	1000	15	985	3.00%	10.37%	97.00%	79.11%	17.89%
(618.716, 654.116]	1000	10	990	2.00%	10.42%	99.00%	89.53%	9.47%
(654.116, 852.11]	1000	5	995	1.00%	10.47%	100.00%	100.00%	0.00%
all	10000	500	9500	100.00%	100.00%			35.79%

← KS最大

3. KS应用和评估标准

1）应用对象

KS指标可以应用在模型或规则上，一般用于模型上较多，规则上通常看Lift提升度。

当应用于模型时，KS可以辅助设定cutoff阈值。理论上可以将分箱的KS最大值对应的取值作为cutoff，但实际还需要考虑对通过率的影响，二者之间做个平衡，所以一般不一定会设置在理想值。

2）评估标准

KS的取值范围是[0,1]，可以用小数或者以%形式的表示，比如KS=0.3或者30%。通常来说，KS越大代表好坏客户的区分度越高。具体多少算高要看使用场景，就贷前/中/后的模型来说，KS也有不同的衡量标准。

贷前A卡模型通常KS大于0.3就视为有较强的区分度了，但一般至少要求0.2以上，0.2以下不建议使用；贷中B卡模型的KS一般要求大于0.4以上；贷后C卡模型的KS一般要求大于0.5以上。如果是子模型或者单一维度模型，可以适当下降标准。

另外，如果KS值高的离谱，比如贷前A卡模型高于50以上，需要怀疑模型是否存在过拟合，或者数据泄露。

4. 模型上线后KS评估

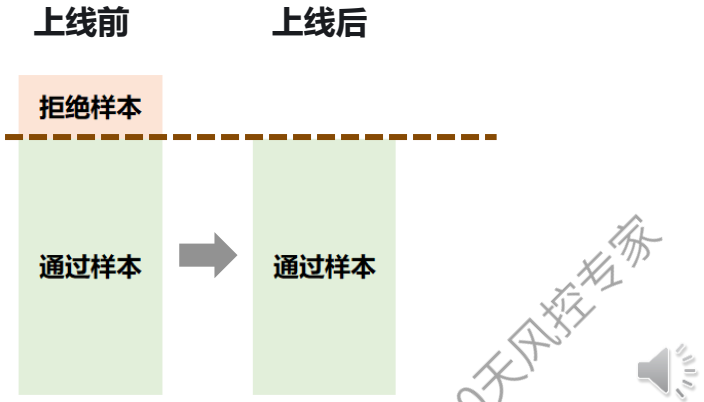
模型上线后KS一定下降，这属于正常现象。

① 为什么KS会下降?

由于模型上线后只能观测到“通过的样本”来计算KS，相对于离线分析的整体样本而言（通过和拒绝样本）是缺少了拒绝样本的，也就是**上线前后统计的样本分布是发生了变化的，这就导致了KS值的不同。**

那具体下降的原因是什么？**原因在于，拒绝样本的坏客户浓度高于通过样本的坏客户浓度。**离线分析时，因为拒绝样本坏客户浓度高时，好坏的区分相对更简单，也就导致KS会更高。而一旦拒绝样本不在了（在线上被拒掉了），好坏的区分能力也就下降了。

xgb_score	sum	bad_count	good_count	badpct	goodpct	cum_badpct	cum_goodpct	ks
(205.388, 448.898]	1000	150	850	30.00%	8.95%	30.00%	8.95%	21.05%
(448.898, 483.828]	1000	100	900	20.00%	9.47%	50.00%	18.42%	31.58%
(483.828, 508.842]	1000	70	930	14.00%	9.79%	64.00%	28.21%	35.79%
(508.842, 530.35]	1001	50	951	10.00%	10.01%	74.00%	38.22%	35.78%
(530.35, 550.93]	1001	45	956	9.00%	10.06%	83.00%	48.28%	34.72%
(550.93, 570.996]	998	35	963	7.00%	10.14%	90.00%	58.42%	31.58%
(570.996, 592.425]	1000	20	980	4.00%	10.32%	94.00%	68.74%	25.26%
(592.425, 618.716]	1000	15	985	3.00%	10.37%	97.00%	79.11%	17.89%
(618.716, 654.116]	1000	10	990	2.00%	10.42%	99.00%	89.53%	9.47%
(654.116, 852.11]	1000	5	995	1.00%	10.47%	100.00%	100.00%	0.00%
all	10000	500	9500	100.00%	100.00%			35.79%



4. 模型上线后KS评估

xgb_score	sum	bad_count	good_count	badpct	goodpct	cum_badpct	cum_goodpct	ks
(205.388, 448.898]	1000	150	850	30.00%	8.95%	30.00%	8.95%	21.05%
(448.898, 483.828]	1000	100	900	20.00%	9.47%	50.00%	18.42%	31.58%
(483.828, 508.842]	1000	70	930	14.00%	9.79%	64.00%	28.21%	35.79%
(508.842, 530.35]	1001	50	951	10.00%	10.01%	74.00%	38.22%	35.78%
(530.35, 550.93]	1001	45	956	9.00%	10.06%	83.00%	48.28%	34.72%
(550.93, 570.996]	998	35	963	7.00%	10.14%	90.00%	58.42%	31.58%
(570.996, 592.425]	1000	20	980	4.00%	10.32%	94.00%	68.74%	25.26%
(592.425, 618.716]	1000	15	985	3.00%	10.37%	97.00%	79.11%	17.89%
(618.716, 654.116]	1000	10	990	2.00%	10.42%	99.00%	89.53%	9.47%
(654.116, 852.11]	1000	5	995	1.00%	10.47%	100.00%	100.00%	0.00%
all	10000	500	9500	100.00%	100.00%			**35.79%**

①
②

模型离线分析设定cutoff，
ks为：0.3579

xgb_score	sum	bad_count	good_count	badpct	goodpct	cum_badpct	cum_goodpct	ks
(448.898, 483.828]	1000	100	900	28.57%	10.40%	28.57%	10.40%	18.17%
(483.828, 508.842]	1000	70	930	20.00%	10.75%	48.57%	21.16%	27.42%
(508.842, 530.35]	1001	50	951	14.29%	10.99%	62.86%	32.15%	30.71%
(530.35, 550.93]	1001	45	956	12.86%	11.05%	75.71%	43.20%	32.51%
(550.93, 570.996]	998	35	963	10.00%	11.13%	85.71%	54.34%	31.38%
(570.996, 592.425]	1000	20	980	5.71%	11.33%	91.43%	65.66%	25.76%
(592.425, 618.716]	1000	15	985	4.29%	11.39%	95.71%	77.05%	18.66%
(618.716, 654.116]	1000	10	990	2.86%	11.45%	98.57%	88.50%	10.07%
(654.116, 852.11]	1000	5	995	1.43%	11.50%	100.00%	100.00%	0.00%
all	9000	350	8650	100.00%	100.00%			**32.51%**

cutoff设定在①，上线后
ks为：0.3251

xgb_score	sum	bad_count	good_count	badpct	goodpct	cum_badpct	cum_goodpct	ks
(483.828, 508.842]	1000	70	930	28.00%	12.00%	28.00%	12.00%	16.00%
(508.842, 530.35]	1001	50	951	20.00%	12.27%	48.00%	24.27%	23.73%
(530.35, 550.93]	1001	45	956	18.00%	12.34%	66.00%	36.61%	29.39%
(550.93, 570.996]	998	35	963	14.00%	12.43%	80.00%	49.03%	30.97%
(570.996, 592.425]	1000	20	980	8.00%	12.65%	88.00%	61.68%	26.32%
(592.425, 618.716]	1000	15	985	6.00%	12.71%	94.00%	74.39%	19.61%
(618.716, 654.116]	1000	10	990	4.00%	12.77%	98.00%	87.16%	10.84%
(654.116, 852.11]	1000	5	995	2.00%	12.84%	100.00%	100.00%	0.00%
all	8000	250	7750	100.00%	100.00%			**30.97%**

cutoff设定在②，上线后
ks为：0.3097

扫码进专属群

4. KS计算的Python代码

```
# 等频分箱
n = 10
df_bin=pd.qcut(df['xgb_score'],n)
group_all=df['target'].groupby(df_bin).count()
group_bad=df['target'].groupby(df_bin).sum()
group_good=group_all-group_bad
var_bin=pd.DataFrame()
var_bin['sum']=group_all
var_bin['bad_count']=group_bad
var_bin['good_count']=group_good

var_bin['badpct']=var_bin['bad_count']/var_bin['bad_count'].sum()
var_bin['goodpct']=var_bin['good_count']/var_bin['good_count'].sum()
var_bin['cum_badpct'] = var_bin['badpct'].cumsum()
var_bin['cum_goodpct'] = var_bin['goodpct'].cumsum()
var_bin['ks'] = round(abs(var_bin['cum_badpct']-var_bin['cum_goodpct']),4)
```

计算累计坏样本占比、累计好样本占比

KS = abs(累计坏样本占比-累计好样本占比)

```
var_bin.style.bar(subset=['ks'], color='orange', width=80, height=80, align='left', vmin=0)
```

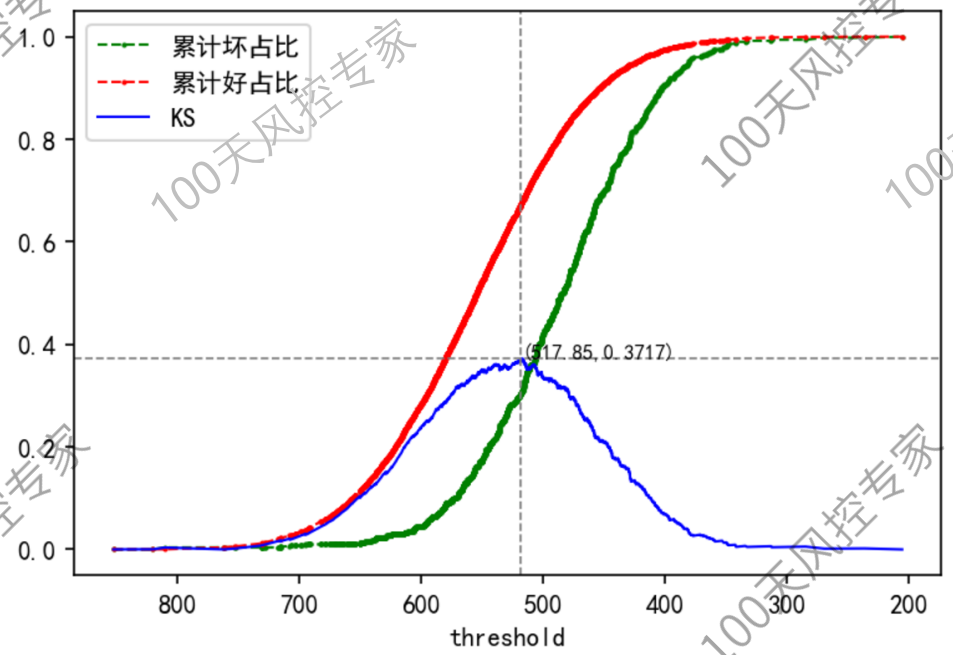
xgb_score	sum	bad_count	good_count	badpct	goodpct	cum_badpct	cum_goodpct	ks
(205.38899999999998, 448.898]	1000	150	850	0.300000	0.089474	0.300000	0.089474	0.210500
(448.898, 483.828]	1000	100	900	0.200000	0.094737	0.500000	0.184211	0.315800
(483.828, 508.842]	1000	70	930	0.140000	0.097895	0.640000	0.282105	0.357900
(508.842, 530.35]	1001	50	951	0.100000	0.100105	0.740000	0.382211	0.357800
(530.35, 550.93]	1001	45	956	0.090000	0.100632	0.830000	0.482842	0.347200
(550.93, 570.996]	998	35	963	0.070000	0.101368	0.900000	0.584211	0.315800
(570.996, 592.425]	1000	20	980	0.040000	0.103158	0.940000	0.687368	0.252600
(592.425, 618.716]	1000	15	985	0.030000	0.103684	0.970000	0.791053	0.178900
(618.716, 654.116]	1000	10	990	0.020000	0.104211	0.990000	0.895263	0.094700
(654.116, 852.11]	1000	5	995	0.010000	0.104737	1.000000	1.000000	0.000000

← KS最大

4. KS可视化

```
######################## KSplot ############################
def ks_simple_plot(data, y, x):
    from sklearn import metrics
    fpr, tpr, thres = metrics.roc_curve(data[y], data[x], pos_label=1)
    ks = abs(tpr[1:]-fpr[1:])
    max_thres = thres[np.argmax(ks)]
    fig = plt.figure(dpi=150)
    plt.plot(thres[1:], tpr[1:], 'go--', linewidth=1, markersize=1)
    plt.plot(thres[1:], fpr[1:], 'ro--', linewidth=1, markersize=1)
    plt.plot(thres[1:], ks, color='blue', linewidth=1, markersize=2)
    plt.axvline(max_thres, color='grey', linestyle='--', linewidth=0.8, label='cutoff')
    plt.axhline(max(ks), color='grey', linestyle='--', linewidth=0.8, label='max_ks')
    plt.text(x=max_thres, y =max(ks), s=('('+str(max_thres)+','+str(round(max(ks),4))+')'), fontsize=8)
    plt.xlabel('threshold')
    plt.legend(['累计坏占比', '累计好占比', 'KS'])
    plt.gca().invert_xaxis()
    plt.show()
```

```
ks_simple_plot(df,'target','xgb_score')
```



Python数据科学

谢谢

扫码进专属群

出品人：东哥起飞

解锁风控课程

关注我的公众号